

שאלה 1 (35 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות. השאלה מתייחסת לגרסת השפה עם מימוש באמצעות מבנה נתונים.

בשאלה זו נרצה להוסיף לשפת "וישגור" יכולת עבודה עם enum בדומה לקיים ברוב שפות התכנות. לשם כך נאמץ צורת עבודה עם enum בדומה לקיים בשפת התכנות RUST (אין צורך בידע מוקדם בשפה זו).

הגדרת enum מורכבת מאוסף של שמות לבחירתו.

נוסיף לשפה, ביטוי המחזיר enum חדש, ביטוי המאפשר להתייחס לאחד השמות בתוך enum קיים, וביטוי המאפשר לבחון ולבצע פעולות לפי ערכי ה-enum.

להלן הדקדוקים המגדירים את 3 היכולות החדשות הנ"ל, לאחריהם תמצאו הסבר על כל אחד מהם וכן הדגמות שימוש.

$Expression ::= enum \{ \{Identifier\}^{+,(*)} \}$

enum-exp (ids)

$Expression ::= < Expression . Identifier >$

enum-elmt-exp (enm id)

$Expression ::= match [Identifier :: Expression]$
 $\{ ?identifier \Rightarrow Expression ; \}^+$

match-exp (enmid exp1 enmids exps)

- ביטוי enum-exp בנוי מרשימה של לפחות שם אחד (אם לא מופיעים שמות יש להדפיס הודעת שגיאה מתאימה). השמות מייצגים את ערכי ה-enum השונים. הביטוי מחזיר כתשובה ייצוג של enum עם השמות שצוינו בהגדרתו. שימו לב, שיש כאן סוג חדש של תשובה מוחזרת.

- ביטוי enum-elmt-exp בנוי מביטוי enm ומשם id. enm הוא ביטוי שתוצאתו היא מסוג enum, ו-id הוא שם שאמור לייצג את אחד הערכים ב-enm. במקרה ש-enm אינו מייצג enum או ש-id אינו חלק מאותו enum יש להדפיס שגיאה מתאימה. במקרה שהכל תקין יש להחזיר כערך את השם ש-id מייצג. גם כאן יש סוג תשובה חדשה מוחזרת.

- ביטוי match מורכב מ-
 - *enum apply* `enumid` שהוא שם מזהה המייצג `enum` שהוגדר קודם לכן. אם זה לא מייצג `enum` יש להדפיס הודעת שגיאה מתאימה
 - `expl` – ביטוי שתוצאתו שם מזהה של אחד מערכי ה-`enum` המיוצג ע"י `enumid`. יש לוודא שהשם הוא אכן אחד מהערכים של ה-`enum`, אחרת יש להדפיס הודעת שגיאה מתאימה.
 - `enmids` רשימה של כל הערכים השונים של ה-`enum` המיוצג ע"י `enumid`. יש לוודא שאכן הרשימה מכילה את כל הערכים שהוגדרו ל-`enumid` וכן שאין ערכים שלא שייכים אליו. במקרים כאלה יש להדפיס הודעת שגיאה מתאימה.
 - `exps` – רשימת ביטויים, אחד לכל ערך ב-`enmids`. מרשימה זו יוחזר בסופו של דבר ערכו של הביטוי המתאים לערך מ-`enmids` השווה לערכו של `expl`.

להלן מספר דוגמאות להמחשת השימוש וההתנהגות של הביטויים החדשים לעבודה עם `enum`:

דוגמה 1:

```
> (run "let Colors = enum { Red, Green, Blue}
    in
      let c = <Colors.Green>
      in
        match [Colors::c]
          ?Red    => 10;
          ?Green  => 100;
          ?Blue   => 1000;
        ")
(num-val 100)
>
```

בדוגמה זו, מוגדר `enum` בשם `Colors` המייצג את הצבעים `Red Green Blue` לאחר מכן מוגדר משתנה `c` המקבל כערך את אחד מערכי ה-`enum` במקרה זה `Green`. לאחר מכן, מבוצע ביטוי `match` על `c` ומאחר והוא מתאים למקרה של `Green` הוחזר 100.

המשך הדוגמאות בעמוד הבא

num-val

(num = list of (num-val))

(enum-val
(in enum?)
(identifier-val
(is string?))

define enum?
list of identifier-val
=
equal

do cases for each until null

דוגמה 2:

```
> (run "let Colors = enum { Red, Green, Blue}
    in
      let c= <Colors.Green>
      in
        c")
(identifier-val 'Green)
>
```

דוגמה זו, דומה לדוגמה הקודמת, ובה מוחזר ערכו של c שכפי שניתן לראות הוא מסוג חדש שאין בשפת "וישגור"

דוגמה 3:

```
> (run "let Colors = enum { Red, Green, Blue}
    in
      let c= <Colors.Green>
      in
        match [Colors::c]
          ?Red    => 10;
          ?Green  => 100;
        ")
```

✖ interp.scm:108:39: match-exp: match cases is not correspond to enum Colors cases

דוגמה זו דומה לדוגמאות קודם, וממחישה שימוש שגוי ב-match. בדוגמה ניתן לראות שבביטוי match חסרה בדיקה לערך Blue ועל כן, הודפסה שגיאה מתאימה

דוגמה 4:

```
> (run "let Colors = enum { Red, Green, Blue}
    in
      let c= <i.Green>
      in
        c")
```

✖ interp.scm:93:32: enum-elmt-exp: not an enum

בדוגמה זו, יש שימוש שגוי בביטוי enum-elmt-exp כפי שניתן לראות יש ניסיון לחלץ את הערך Green מ-i שאינו מייצג enum כלל (הוא מייצג מספר, הוא אחד המשתנים הקיימים בסביבה ההתחלתית של המפרש)

דוגמה 5:

```
> (run "let Colors = enum { Red, Green, Blue}
    in
      let c= <Colors.Yellow>
      in
        c")
interp.scm:91:42: enum-elmt-exp: Yellow not found
> |
```

בדוגמה זו ניתן לראות שיש ניסיון לחלץ מ- Colors ערך שלא מצוי בו (Yellow) ועל כן התקבלה שגיאה מתאימה.

דוגמה 6:

```
> (run "let Colors = enum { }
    in
      let c= <Colors.Red>
      in
        c")
interp.scm:80:22: enum-exp: empty identifier list in enum definition
> |
```

בדוגמה זו, יש הגדרת enum ללא ערכים, ולכן התקבלה שגיאה מתאימה

דוגמה 7:

```
> (run "let Foods = enum {Banana, Humus, Hamburger}
    in
      let Calories= proc (x)
        match [Foods::x]
        [ ?Banana => 90;
          ?Humus   => 170;
          ?Hamburger => 300;
        ]
      in
        (Calories <Foods.Humus>) ")
(num-val 170)
>
```

$3-2=1$

expression

בדוגמה זו, מוגדר enum המייצג מאכלים שונים, וכן מוגדרת פרוצדורה בשם Calories המקבלת פרמטר שאמור לייצג מאכל כלשהו (מרשימת המאכלים של ה- enum Foods) ולהחזיר את כמות הקלוריות המתאימה לו.

לבסוף מופעלת הפרוצדורה Calories עם הפרמטר המייצג את המאכל Humus (חמוס) וערכה של כל התוכנית הוא 170 כפי שחושב ע"י הפרוצדורה.

ממשו והטמיעו את השינויים הדרושים כפי שהוגדרו והודגמו בתוך שפת **"וישגור" (שפת PROC)**.

בתשובתכם, הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות בקבצי המפרש, **ומהם** השינויים והתוספות.

לא תתקבל תשובה של תיאור מילולי. יש לממש בקוד את השינויים הדרושים.

הנחיות כלליות לפתרון:

- א. הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- ב. הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- ג. הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
- ד. על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
- ה. בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- ו. ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב-terminals, non-terminals ופעולות של סגור ($\{ \}^*$ או $\{ \}^+$)

המשך בחינה בעמוד הבא

שאלה 2 (35 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS).

בשאלה זו נרצה לממש ביטוי חדש, המספק לולאה בעזרתה ניתן לבנות פרוצדורות שונות. לולאה יש משתנה ייעודי, ובכל איטרציה של הלולאה, נבנית עבורו פרוצדורה שונה, בה ניתן להשתמש בגוף הלולאה. לאיטרציות הלולאה יכולים להיות תנאים מקדימים (guards), בעזרתם ניתן לשלוט על הפסקת הלולאה טרם הזמן, או דילוג על ביצוע איטרציות מסוימות.

לשם כך נרחיב את שפת "וירמוז" עם ביטוי חדש בשם for/proc.

להלן הדקדוק המגדיר את תחביר הביטוי החדש ולאחריו הסבר על התנהגותו:

```
Expression ::= for/proc ( identifier ;  
                        [ { identifier }(*) ] ;  
                        [ { Expression }(*) ] )  
                        { < Guard > }*  
                        {  
                          Expression  
                        }
```

forproc-exp (Id Ids Bodies Guards Body)

Guard ::= skip : Expression

skip-guard (exp)

Guard ::= break : Expression

break-guard (exp)

הסבר על הדקדוק מופיע בעמוד הבא

הסבר הדקדוק והתנהגות הביטוי:

- לביטוי forproc המרכיבים הבאים:
 Id - משתנה ייעודי חדש לצורך ביצוע הלולאה, אינו מוכר לאחר סיום הלולאה.
 - Ids - Bodies - רשימות של שמות מזהים, ושל ביטויים בהתאמה (הרשימות אמורות להיות באותו האורך – יש לוודא זאת). בכל איטרציה משתנה הלולאה id יהיה תוצאת בניה של פרוצדורה שמקבלת פרמטר ששמו ייקבע לפי האיבר הבא מהרשימה ids וגוף הפרוצדורה שתיבנה ייקבע לפי הביטוי הבא ברשימה bodies.
 - Guards - היא רשימה של איברים מסוג Guard. קיימים 2 סוגי Guard: skip ו-break. כל אחד מהם מורכב מביטוי, שערכו אמור להיות בוליאני. ב-Guard מסוג skip כאשר התנאי הבוליאני הוא אמת, יבוצע דילוג על האיטרציה הנוכחית, והלולאה תעבור לאיטרציה הבאה. ב-Guard מסוג break, כאשר התנאי הבוליאני הוא אמת, פעולת הלולאה תיפסק. בכל איטרציית לולאה, תחילה נבנית הפרוצדורה הבאה למשתנה הלולאה, לאחר מכן, נבדקים ה-Guards השונים, ועל פיהם נקבע אם גוף הלולאה יבוצע באותה איטרציה.
 - Body - הוא גוף הלולאה שיש לבצע בכל איטרציה.
- הלולאה עוצרת כאשר, מתקיים Guard מסוג break, או כאשר לא נותרו איברים ברשימות Ids ו-Bodies.

לביטוי forproc-exp אין ערך מוחזר, ולכן יש להחזיר ערך פיקטיבי של 27 num-val

נדרש לבדוק ולהדפיס הודעות שגיאה במקרים של שימוש לא תקין בביטוי.

להלן דוגמה+ הסבר להמחשת השימוש בביטוי החדש:

```
> (run "let w1=100
  in
    let w2=300
    in
      let w3=500
      in
        begin
          for/proc (p; [a,b,c,d]; [-(a,1), -(b,2), -(c,3), -(d,4)])
            <skip: zero? ((p 2))>
            <break: zero? ((p 12)) >
            <skip: zero? ((p 200))>
          {
            if zero? ((p 1))
            then
              set w1 = 7000
            else
              if zero? ((p 3))
              then
                set w2 = p
              else
                set w3=(p w1)
          };
          (w2 w3)
        end")
(num-val 6993)
> |
```

בדוגמה זו, מוגדרים 3 משתנים $w1, w2, w3$ המאותחלים לערכים 100, 300, 500 בהתאמה. לאחר מכן, מבוצעת לולאת for/proc בעלת משתנה לולאה בשם p. בלולאה זו נבנות 4 פרוצדורות (בכל איטרציה נבנית פרוצדורה אחרת).

באיטרציה הראשונה p מייצג פרוצדורה בעלת פרמטר בשם a וגוף פרוצדורה המבצע $-(a, 1)$ מאחר ואף אחד מה-Guards לא מתקיימים, מבוצע גוף הלולאה המשנה את ערכו של w1 להיות 7000.

באיטרציה השנייה של הלולאה, משתנה הלולאה p מייצג פרוצדורה בעלת פרמטר בשם b וגוף פרוצדורה המבצע $-(b, 2)$ מאחר וה-Guard מתקיים, יש דילוג על איטרציה זו ומעבר לאיטרציה השלישית.

באיטרציה השלישית, p מייצג פרוצדורה עם פרמטר בשם c וגוף פרוצדורה המבצע $-(c, 3)$, אף אחד מה-Guards לא מתקיים, ולכן מבוצע גוף הלולאה, שבמקרה זה גורם למשתנה w2 להיות שווה לפרוצדורה הנוכחית ש-p מייצג.

באיטרציה הרביעית, p מייצג פרוצדורה עם פרמטר בשם d וגוף פרוצדורה המבצע $-(d, 4)$, אף אחד מה-Guards לא מתקיים, ולכן מבוצע גוף הלולאה, המעדכן את ערכו משתנה w3 להיות 6996.

בשלב זה הלולאה מסתיימת, ולאחריה מבוצע הביטוי $(w2 \ w3)$ שגורם להפעלת הפרודורה ש-w2 מייצג (המחסירה 3 מהפרמטר שמועבר לה). ועל כן ערכה של כל התוכנית הוא 6993 דוגמה שונה מעט נוספת:

```
> (run "let w1=100
  in
    let w2=300
    in
      let w3=500
      in
        begin
          for/proc (p; [a,b,c,d]; [-(a,1), -(b,2), -(c,3), -(d,4)])
            < break: zero? ((p 2)) >
            < break: zero? ((p 12)) >
            < skip: zero? ((p 200)) >
          {
            if zero? ((p 1))
            then
              set w1 = 7000
            else
              if zero? ((p 3))
              then
                set w2 = p
              else
                set w3=(p w1)
          };
          -(w1,w2)
        end")
(num-val 6700)
>
```


בדוגמה זו, הדומה לדוגמה הקודמת עם מעט שינויים, הלולאה מתבצעת פעם אחת בלבד, ומספיקה לשנות את ערכו של w_1 ל-7000, באיטרציה השניה ה-Guard מתקיים ולכן, הלולאה מסתיימת (מאחר וזה break guard), לאחר סיום הלולאה מבוצע הביטוי (w_1, w_2) – שערכו הוא 6700.

ממשו והטמיעו את הביטוי החדש בתוך שפת "זירמוז" (שפת IMPLICIT-REFS).

בתשובתכם, הקפידו להסביר היכן בדיוק נדרשים שינויים ותוספות בקבצי המפרש, ומהם השינויים והתוספות.

יש לשים לב ולבדוק מקרים של שימוש לא תקין בתחביר.
במקרים של שימוש לא תקין, יש להדפיס הודעות שגיאה מתאימות.

הנחיות כלליות לפתרון:

- 1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- 2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- 3) הפתרון אמור לשמור על הגדרות השפה המקוריות. (כגון אופן ניהול הסביבה וסוגי הערכים בה, אופן ניהול ההחסן והערכים שניתן לשמור בו, וכדומה)
- 4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכן)
- 5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- 6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור (} *) או { + }

המשך בחינה בעמוד הבא

שאלה 3 (30 נקודות)

להלן תוכנית בשפת PROC (שפת "וישגור")

```
let p = proc (a) - (a, - (0, a))
in
  let a=9
  in
    - (6, - (0, 6)) 12 a+d
    (if (let a= 6
          in zero? (- ((p f a), a) ))
      then a x y
      else (p y a) 18)
```

תוצאת הריצה של תוכנית זו היא :

- (num-val 12) א
- (num-val 18) ב
- (num-val 6) ג
- (num-val 9) ד
- ה) התוכנית לא תקינה תחבירית
- ו) התוכנית מחזירה שגיאה בזמן הריצה.

נמקו והסבירו את תשובתכם ע"י הסבר של דרך חישוב תוצאת התוכנית

את תשובתכם רשמו במחברת הבחינה בלבד.

בהצלחה !